```
        memory_order success_order,memory_order failure_order)
noexcept;

    operator T*() const volatile noexcept;
    operator T*() const noexcept;

    T* fetch_add(
        ptrdiff_t,memory_order = memory_order_seq_cst) volatile
noexcept;
    T* fetch_add(
        ptrdiff_t,memory_order = memory_order_seq_cst) noexcept;
    T* fetch_sub(
        ptrdiff_t,memory_order = memory_order_seq_cst) volatile
noexcept;
    T* fetch_sub(
        ptrdiff_t,memory_order = memory_order_seq_cst) noexcept;

    T* operator++() volatile noexcept;
    T* operator++() noexcept;
    T* operator++(int) volatile noexcept;
    T* operator++(int) noexcept;
    T* operator--() volatile noexcept;
    T* operator--() noexcept;
    T* operator--(int) volatile noexcept;
    T* operator--(int) noexcept;

    T* operator+=(ptrdiff_t) volatile noexcept;
    T* operator+=(ptrdiff_t) noexcept;
    T* operator-=(ptrdiff_t) volatile noexcept;
    T* operator-=(ptrdiff_t) noexcept;
};

bool atomic_is_lock_free(volatile const atomic<T*>*) noexcept;
bool atomic_is_lock_free(const atomic<T*>*) noexcept;
void atomic_init(volatile atomic<T*>*, T*) noexcept;
void atomic_init(atomic<T*>*, T*) noexcept;
T* atomic_exchange(volatile atomic<T*>*, T*) noexcept;
T* atomic_exchange(atomic<T*>*, T*) noexcept;
T* atomic_exchange_explicit(volatile atomic<T*>*, T*, memory_order)
    noexcept;
T* atomic_exchange_explicit(atomic<T*>*, T*, memory_order)
noexcept;
void atomic_store(volatile atomic<T*>*, T*) noexcept;
void atomic_store(atomic<T*>*, T*) noexcept;
void atomic_store_explicit(volatile atomic<T*>*, T*, memory_order)
    noexcept;
void atomic_store_explicit(atomic<T*>*, T*, memory_order) noexcept;
T* atomic_load(volatile const atomic<T*>*) noexcept;
T* atomic_load(const atomic<T*>*) noexcept;
T* atomic_load_explicit(volatile const atomic<T*>*, memory_order)
```